Noctalia

Control Center Widget

Control center widgets are button components that appear in the Control Center panel. They provide quick access to plugin functionality directly from the shortcuts area.

Basic Structure

A control center widget is typically a simple button that can toggle a panel or perform an action:

```
import QtQuick
import Quickshell
import qs.Widgets

NIconButtonHot {
  property ShellScreen screen
  property var pluginApi: null

  icon: "my-icon"
  tooltipText: "My Plugin"

  onClicked: {
    if (pluginApi) {
      pluginApi.togglePanel(screen, this)
    }
  }
}
```

> **Tip**
>
> You are viewing the **v4 (stable)** documentation. **Switch to v5 (alpha) →**
>
> button, similar to how core Noctalia panels work.

Manifest Entry Point

Add the controlCenterWidget entry point to your manifest.json:

```
{
  "id": "my-plugin",
  "name": "My Plugin",
  "version": "1.0.0",
  "author": "Your Name",
  "description": "A plugin with a control center widget",
  "entryPoints": {
    "controlCenterWidget": "ControlCenterWidget.qml",
    "panel": "Panel.qml"
  }
}
```

Required Properties

Your control center widget component must accept these properties:

Property	Type	Description
screen	ShellScreen	The screen the widget is displayed on
pluginApi	var	The plugin API object (injected by PluginService)

Widget Types

NIconButtonHot (Recommended)

The most common widget type. Displays an icon that can be clicked and shows a tooltip on hover:


```
import QtQuick
import Quickshell
import qs.Widgets

NIconButtonHot {
  property ShellScreen screen
  property var pluginApi: null

  icon: "settings"
  tooltipText: pluginApi?.tr("widget.tooltip") || "Open Settings"

  onClicked: pluginApi?.togglePanel(screen, this)
}
```

NIconButton

A simpler icon button without the hot state styling:

```
import QtQuick
import Quickshell
import qs.Widgets

NIconButton {
  property ShellScreen screen
  property var pluginApi: null

  icon: "refresh"
  tooltip: "Refresh Data"

  onClicked: {
    // Perform an action
    pluginApi?.mainInstance?.refresh()
  }
}
```

Custom Widget


```
import QtQuick
import QtQuick.Layouts
import Quickshell
import qs.Commons
import qs.Widgets

Rectangle {
  id: root

  property ShellScreen screen
  property var pluginApi: null

  implicitWidth: 32
  implicitHeight: 32
  radius: Style.radiusS
  color: mouseArea.containsMouse ? Color.mSurfaceVariant : "transparent"

  NIcon {
    anchors.centerIn: parent
    icon: "star"
    color: Color.mOnSurface
  }

  MouseArea {
    id: mouseArea
    anchors.fill: parent
    hoverEnabled: true
    cursorShape: Qt.PointingHandCursor
    onClicked: pluginApi?.togglePanel(root.screen, root)
  }
}
```

Accessing Plugin Settings

Read settings from your plugin API:


```
  property ShellScreen screen
  property var pluginApi: null

  // Read icon from settings with fallback
  icon: pluginApi?.pluginSettings?.icon || "star"
  tooltipText: pluginApi?.pluginSettings?.tooltipText || "My Widget"

  onClicked: pluginApi?.togglePanel(screen, this)
}
```

Using Translations

Support multiple languages in your widget:

```
NIconButtonHot {
  property ShellScreen screen
  property var pluginApi: null

  icon: "globe"
  tooltipText: pluginApi?.tr("controlCenter.tooltip") || "Open Panel"

  onClicked: pluginApi?.togglePanel(screen, this)
}
```

Translation file (i18n/en.json):

```
{
  "controlCenter": {
    "tooltip": "Open My Plugin"
  }
}
```

Complete Example


```
import QtQuick
import Quickshell
import qs.Widgets

NIconButtonHot {
  property ShellScreen screen
  property var pluginApi: null

  icon: "noctalia"
  tooltipText: "Hello World"

  onClicked: {
    if (pluginApi) {
      pluginApi.togglePanel(screen, this)
    }
  }
}
```

Best Practices

1. **Keep it simple** - Control center widgets should be simple buttons, not complex UIs
2. **Use tooltips** - Always provide a tooltip so users know what the button does
3. **Toggle panels** - The most common pattern is to toggle a panel on click
4. **Use standard icons** - Use Tabler icons that match the Noctalia design language
5. **Handle null pluginApi** - Always check if pluginApi exists before using it

See Also

- Bar Widget - Widgets for the top/bottom bar
- Panel - Create full overlay panels
- Plugin API - Full API reference

- Manifest Reference - Plugin configuration